Virtual File Systems

1 Basic Idea

Virtual File System

A **Virtual File System**, or **VFS**, is an operating-system layer that provides one common interface for many different concrete file systems.

- A computer may use several file systems at the same time.
- Examples include NTFS, FAT, ext2, ext3, ReiserFS, ISO 9660, and network file systems.
- The VFS hides many differences between these concrete file systems.
- User programs call standard file operations such as `open`, `read`, `write`, and `lseek`.
- The VFS decides which underlying file system should handle each request.

2 Windows and UNIX Approaches

Point	Windows Style	UNIX Style
User view	Different file systems often appear as different drive letters, such as `C:` and `D:`.	Multiple file systems are integrated into one directory hierarchy.
Mounting idea	The drive letter identifies the target file system.	File systems are mounted at directories such as `/usr`, `/home`, or `/mnt`.
Visibility	The distinction is more visible to users.	Users see one unified file-system tree.
Example	NTFS on `C:`, FAT on `D:`, DVD as another drive.	ext2 root, ext3 mounted on `/usr`, ReiserFS on `/home`, CD-ROM on `/mnt`.

3 Position of the VFS

- User processes make standard POSIX calls.
- The POSIX interface forms the upper interface of the VFS.
- The VFS layer receives the request first.
- The VFS forwards the request to the correct concrete file system.
- The concrete file systems use their own internal code and buffer cache to perform the real work.

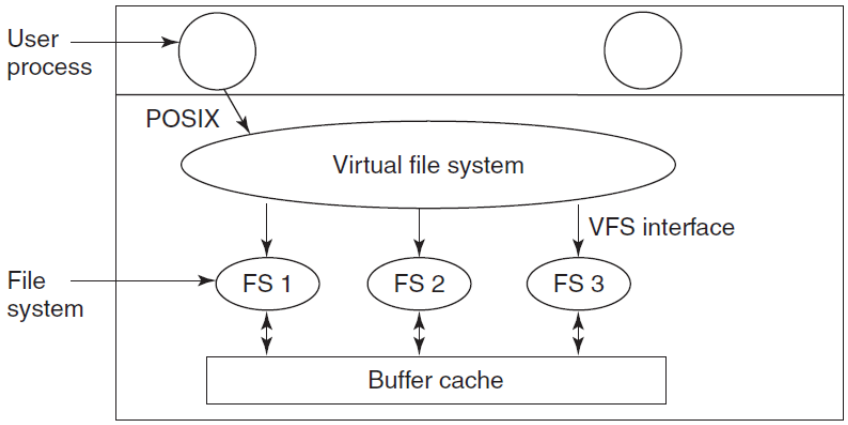

Figure 4-18. Position of the virtual file system.

Figure 1: Position of the virtual file system between POSIX calls and concrete file systems.

4 Two VFS Interfaces

VFS Interfaces

The VFS has an upper interface for user processes and a lower interface for concrete file systems.

Interface	Purpose
Upper interface	Provides standard system calls such as `open`, `read`, `write`, `close`, and `lseek`.
Lower interface	Defines functions that concrete file systems must provide to the VFS.

- The lower interface is sometimes called the **VFS interface**.
- A new file system must implement the functions expected by the VFS.
- These functions may include operations for reading blocks, writing blocks, opening files, reading directories, and managing metadata.
- The VFS does not need to know whether the data is stored on a local disk, USB drive, CD-ROM, or remote server.

5 Important VFS Objects

- VFS implementations are often object-oriented in design, even when written in C.
- A **superblock** describes a mounted file system.
- A **v-node** describes an open or referenced file.
- A **directory object** describes a file-system directory.
- Each object has associated operations supplied by the concrete file system.
- The VFS also maintains internal structures such as the mount table and file-descriptor tables.

6 Registration and Mounting

File-System Registration

When a concrete file system is mounted, it registers with the VFS by providing function pointers for the operations the VFS expects.

1. At boot time, the root file system registers with the VFS.
2. Later, other file systems may be mounted.
3. Each mounted file system provides a table of function addresses.
4. The VFS stores these addresses.
5. When the VFS needs an operation, it calls the correct function through the table.

7 Opening a File Through VFS

Example

Suppose a file system is mounted on `/usr`, and a process calls:

```
open("/usr/include/unistd.h", O_RDONLY)
```

1. The VFS begins parsing the path.
2. It notices that another file system is mounted on `/usr`.
3. It finds that mounted file system's superblock.
4. It starts lookup inside the root directory of the mounted file system.
5. It resolves `include/unistd.h`.
6. It creates a v-node in memory.
7. It asks the concrete file system for the information from the file's i-node.
8. It stores the needed information in the v-node.
9. It creates an entry in the calling process's file-descriptor table.
10. It returns a file descriptor to the process.

8 Reading a File Through VFS

1. The process calls `read` using a file descriptor.
2. The VFS uses the process table and file-descriptor table to find the open-file state.
3. The open-file state points to the v-node.
4. The v-node points to a table of function pointers.
5. The VFS calls the concrete file system's read function.
6. The concrete file system locates and returns the requested data.

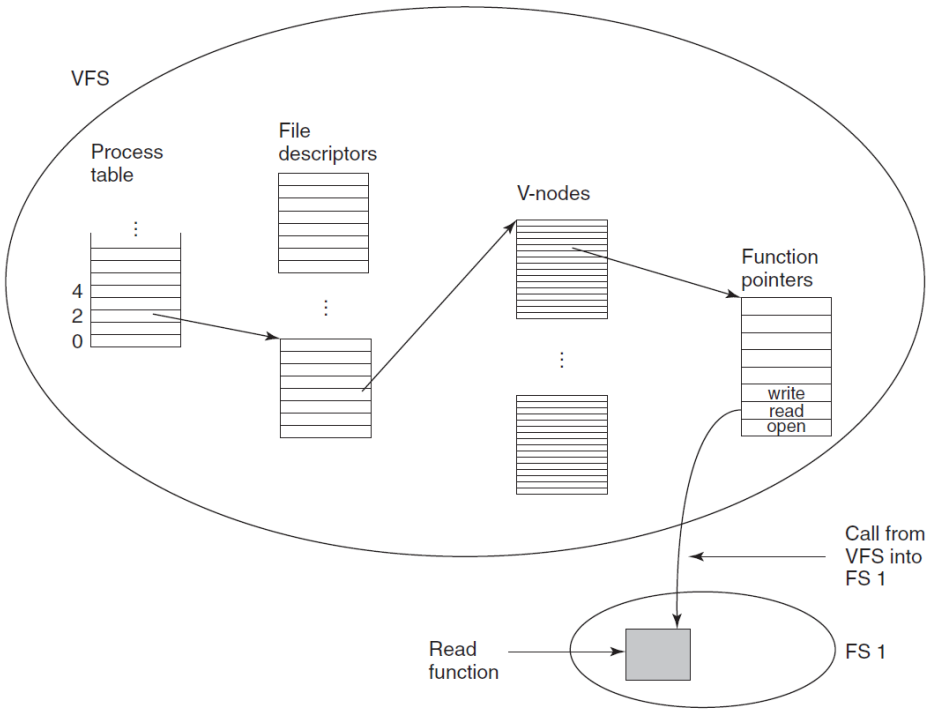

Figure 4-19. A simplified view of the data structures and code used by the VFS and concrete file system to do a read.

Figure 2: Simplified data structures and code path used by the VFS and concrete file system during a read.

9 Why VFS Is Useful

Benefit	Explanation
Uniform API	Programs use the same system calls for different file systems.
Mount integration	Different file systems can appear inside one directory tree.
Extensibility	New file systems can be added by implementing the VFS interface.
Location transparency	The VFS does not care whether data is local, removable, optical, or remote.
Cleaner kernel design	Common file-system behavior is centralized in the VFS layer.

10 Key Points

- VFS abstracts the common part of file-system behavior.
- It receives standard POSIX file calls from user processes.
- It forwards operations to the correct concrete file system.
- Concrete file systems register function tables with the VFS.
- V-nodes represent files at the VFS level.
- Superblocks represent mounted file systems.

- The design makes it easier to support many file systems in one operating system.

Final Point

The VFS layer lets one operating system present a unified file interface while delegating real storage work to many different concrete file systems.